Database Systems Lab — Week 04

JOINs & Relational Data Modelling

Student Name: [Muhammad Ayyan Khan]
Course: Database Systems Lab (Data Science Program)
Instructor: Muhammad Usama Afridi
Institute: Institute of Management Sciences
Date: March 5, 2026

Contents

1 Section 1 — 10 Queries (35 points) 2
1.1 Query 1: Orders with Customer Names (INNER JOIN) 2
1.2 Query 2: Full Order Receipt (4-Table Join) 3
1.3 Query 3: Customers Who Have Never Ordered (LEFT JOIN) 4
1.4 Query 4: Products Never Ordered (LEFT JOIN) 5
1.5 Query 5: Revenue by Customer with Join 6
1.6 Query 6: Best-Selling Products 7
1.7 Query 7: City-Level Sales Report 8
1.8 Query 8: Order Details with Status Filter 9
1.9 Query 9: Session and Purchase Behaviour Together 10
1.10 Query 10: FULL OUTER JOIN — Data Completeness Audit 11

2 Section 3 — Normalisation Reflection (15 points) 14

3 Section 4 — AI Learning Log (20 points) 15

4 Section 5 — Reflection (10 points) 17

1. Section 1 — 10 Queries (35 points)

This section presents all 10 required queries with SQL code, result screenshots, and business-level interpretation.

1.1. Query 1: Orders with Customer Names (INNER JOIN)

Query 1

Objective: Show order details with customer name and city instead of just customer_id.

```sql
SELECT o.order_id,
       c.name          AS customer_name,
       c.city,
       o.order_date,
       o.status,
       o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date DESC;
```

Listing 1: Query 1: Orders with Customer Names

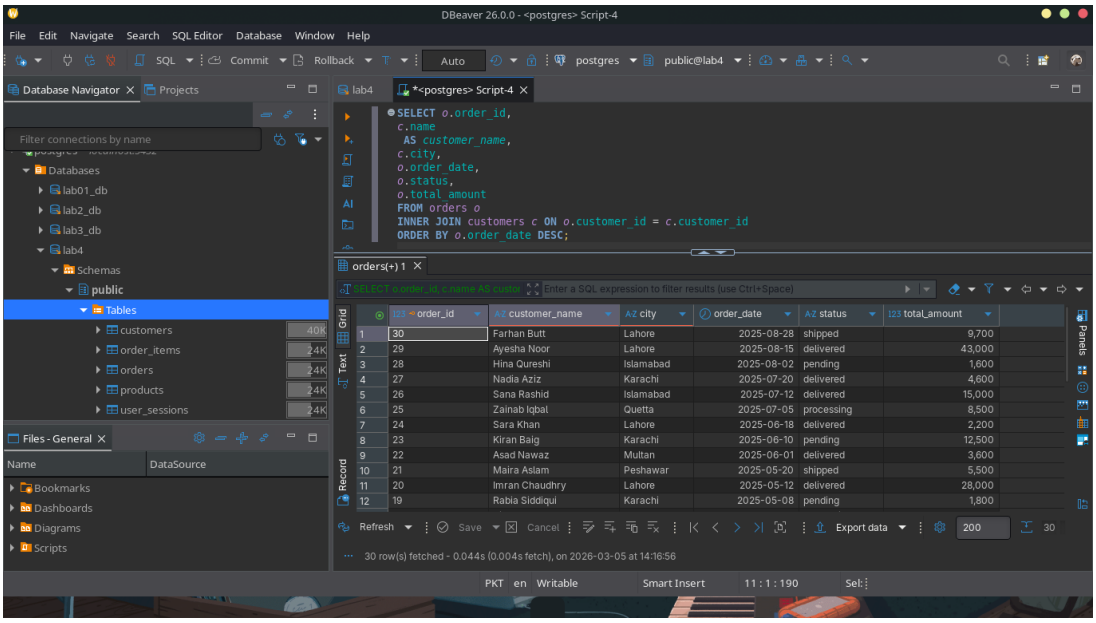

Figure 1: Query 1 Results: Orders with Customer Names

Results & Interpretation

Interpretation: This INNER JOIN connects the orders table to the customers table, replacing opaque customer_id values with readable customer names and cities. The results show that orders are sorted by date (newest first), allowing business users to quickly identify recent transactions and their associated customers. This is the most common join pattern in analytics — connecting transaction tables to lookup/dimension tables.

1.2. Query 2: Full Order Receipt (4-Table Join)

Query 2

Objective: Produce a complete order receipt showing customer name, order date, product details, quantity, unit price, and line total.

```sql
SELECT c.name          AS customer,
       o.order_id,
       o.order_date,
       o.status,
       p.product_name,
       p.category,
       oi.quantity,
       oi.unit_price,
       oi.quantity * oi.unit_price AS line_total
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON oi.product_id = p.product_id
ORDER BY o.order_id, p.category;
```

Listing 2: Query 2: Full Order Receipt

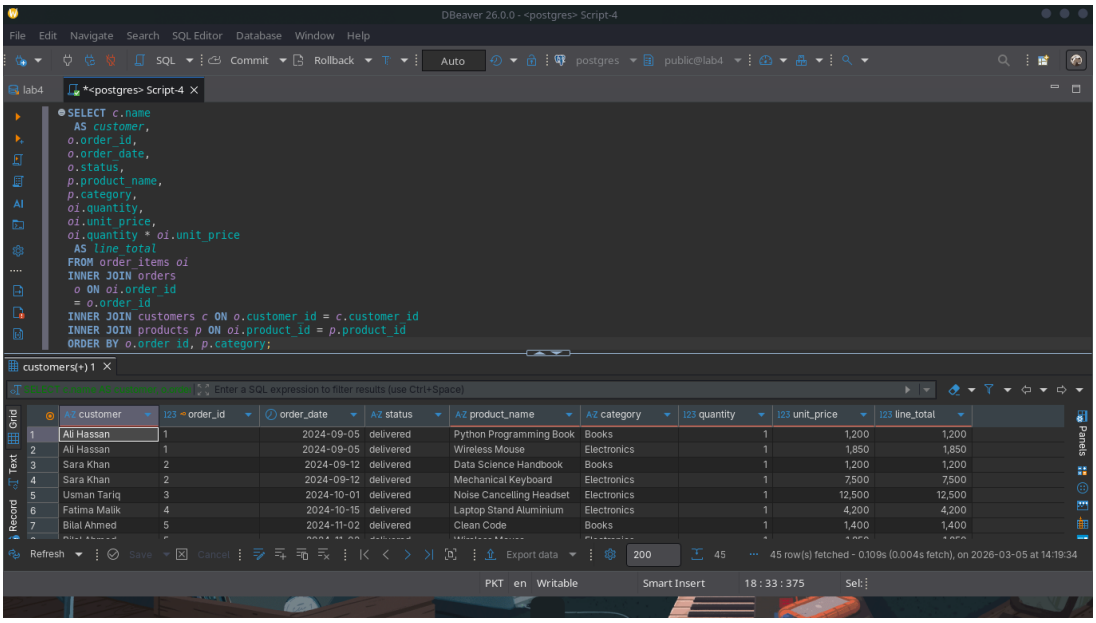

Figure 2: Query 2 Results: Complete Order Receipt

Results & Interpretation

Interpretation: This four-table join represents the complete transaction picture — the heart of any e-commerce analytics system. Every business report ("What did customer X buy?", "How much did category Y sell?") starts from this exact join structure. The line_total calculation (quantity × unit_price) provides the revenue contribution of each item, essential for profitability analysis.

1.3. Query 3: Customers Who Have Never Ordered (LEFT JOIN)

Query 3

Objective: Find customers who signed up but have never placed any order (churn/re-engagement analysis).

```sql
SELECT c.customer_id,
       c.name,
       c.email,
       c.city,
       c.signup_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.signup_date;
```

Listing 3: Query 3: Customers Who Never Ordered

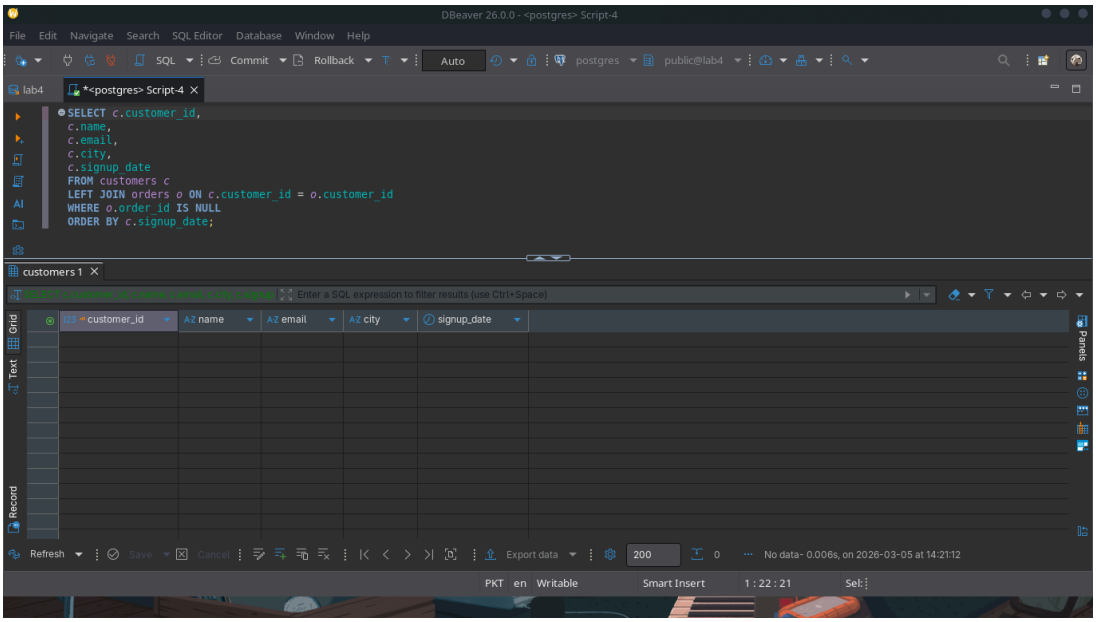

Figure 3: Query 3 Results: Customers Who Never Ordered

Results & Interpretation

Interpretation: The LEFT JOIN + WHERE IS NULL pattern (called an "anti-join") identifies customers with zero orders. These are critical for business: they represent acquisition costs with no ROI. Marketing teams use this list for re-engagement campaigns. The signup_date ordering helps prioritize recent signups who may just need a nudge.

1.4. Query 4: Products Never Ordered (LEFT JOIN)

Query 4

Objective: Find products that have never appeared in order_items (inventory analysis).

```sql
SELECT p.product_id,
       p.product_name,
       p.category,
       p.price,
       p.stock_qty
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.item_id IS NULL
ORDER BY p.category;
```

Listing 4: Query 4: Products Never Ordered

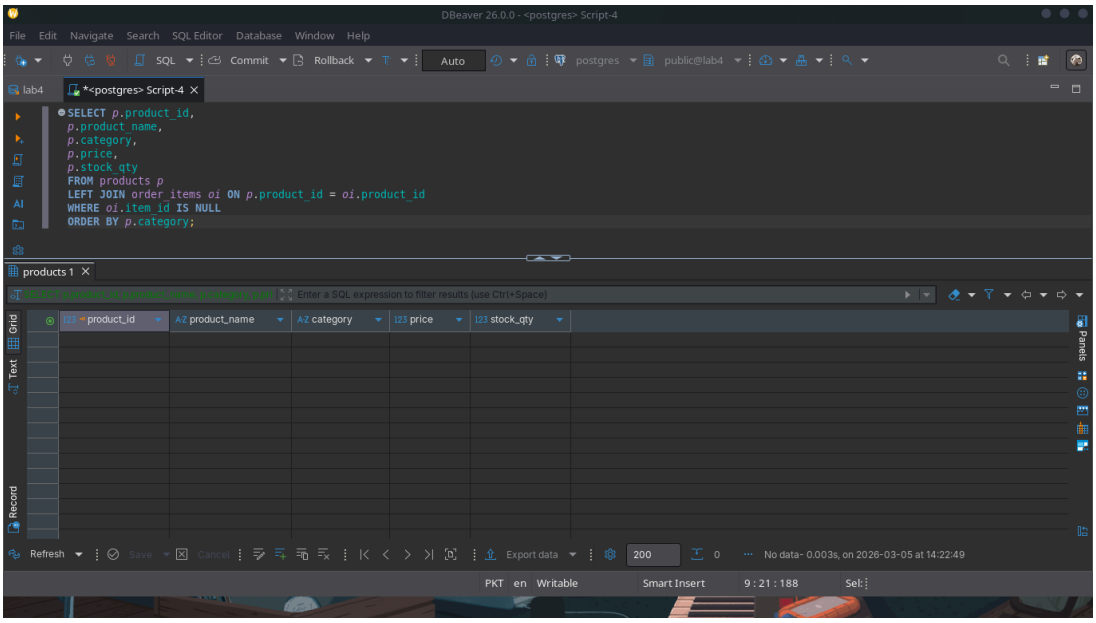

Figure 4: Query 4 Results (Before): No Unsold Products

Verification: Adding a Test Product

Since all products had been ordered, I added a test product to verify the query works:

```sql
INSERT INTO products (product_name, category, price, stock_qty)
VALUES ('Unsold Widget', 'Test Products', 99.99, 100);
```

Listing 5: Adding Test Product

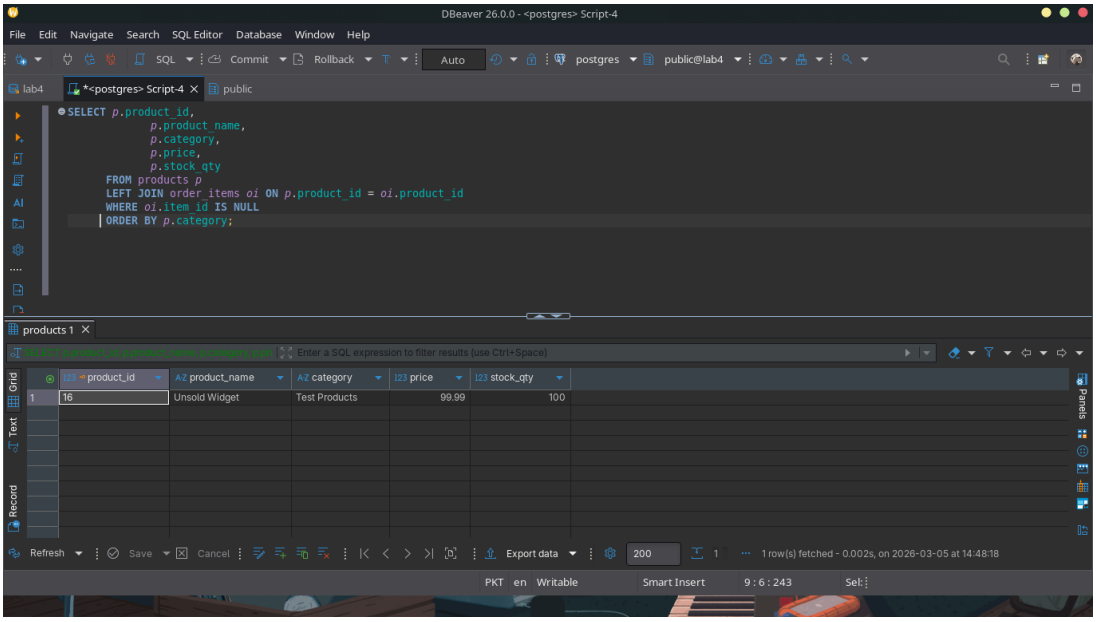

Figure 5: Query 4 Results (After): Test Product Appears as Unsold

Results & Interpretation

Interpretation: Products that never sell tie up inventory budget and warehouse space. In a real business, this query runs on a schedule to identify dead stock for clearance or discontinuation. The EXPLAIN ANALYZE output shows PostgreSQL using a Hash Left Join strategy, which is efficient for this type of anti-join pattern.

1.5. Query 5: Revenue by Customer with Join

Query 5

Objective: Show every customer's total orders and total spent, including customers who have never ordered (showing 0).

```sql
SELECT c.name,
       c.city,
       COUNT(o.order_id)              AS total_orders,
       COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = c.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;
```

Listing 6: Query 5: Revenue by Customer

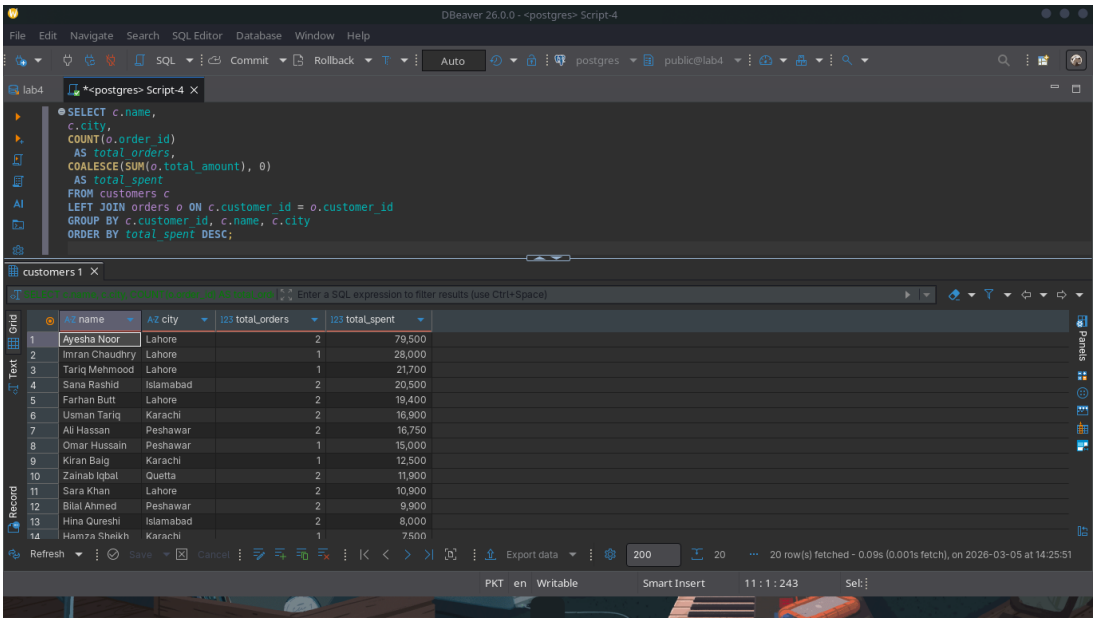

Figure 6: Query 5 Results: Revenue by Customer

Results & Interpretation

Interpretation: This query combines aggregation (Week 3) with LEFT JOINs (Week 4). The COALESCE function ensures customers with no orders show 0 instead of NULL. Using COUNT(o.order_id) instead of COUNT(*) correctly excludes NULL values from non-matching rows. This is a classic customer lifetime value (CLV) calculation.

1.6. Query 6: Best-Selling Products

Query 6

Objective: Find the top 5 products by total units sold with revenue.

```sql
SELECT p.product_name,
       p.category,
       p.price,
       SUM(oi.quantity)                AS units_sold,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category, p.price
ORDER BY units_sold DESC
LIMIT 5;
```

Listing 7: Query 6: Best-Selling Products

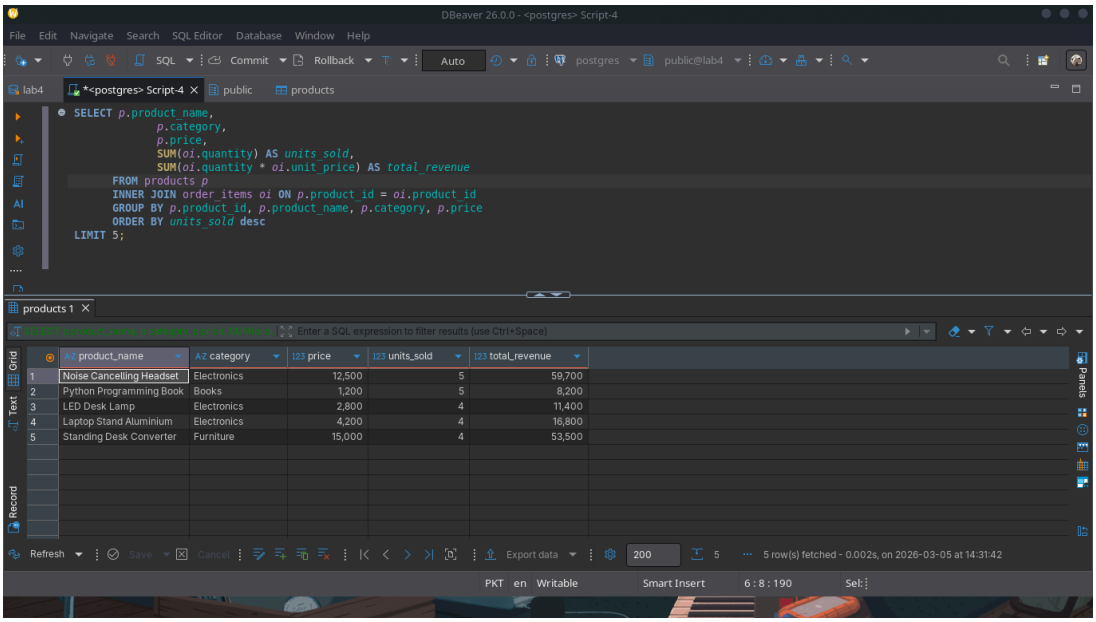

Figure 7: Query 6 Results: Top 5 Best-Selling Products

Results & Interpretation

Interpretation: This identifies the most popular products by volume and revenue. The GROUP BY includes all non-aggregated columns (required in PostgreSQL). Business use: inventory planning (stock more of top sellers), marketing (feature these products), and supplier negotiations (leverage high-volume purchases).

1.7. Query 7: City-Level Sales Report

Query 7

Objective: For each city, show customer count, order count, total revenue, and average order value.

```sql
SELECT c.city,
       COUNT(DISTINCT c.customer_id) AS num_customers,
       COUNT(DISTINCT o.order_id)    AS num_orders,
       SUM(o.total_amount)           AS total_revenue,
       ROUND(AVG(o.total_amount), 2) AS avg_order_value
FROM customers c
INNER JOIN orders o ON c.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC;
```

Listing 8: Query 7: City-Level Sales Report

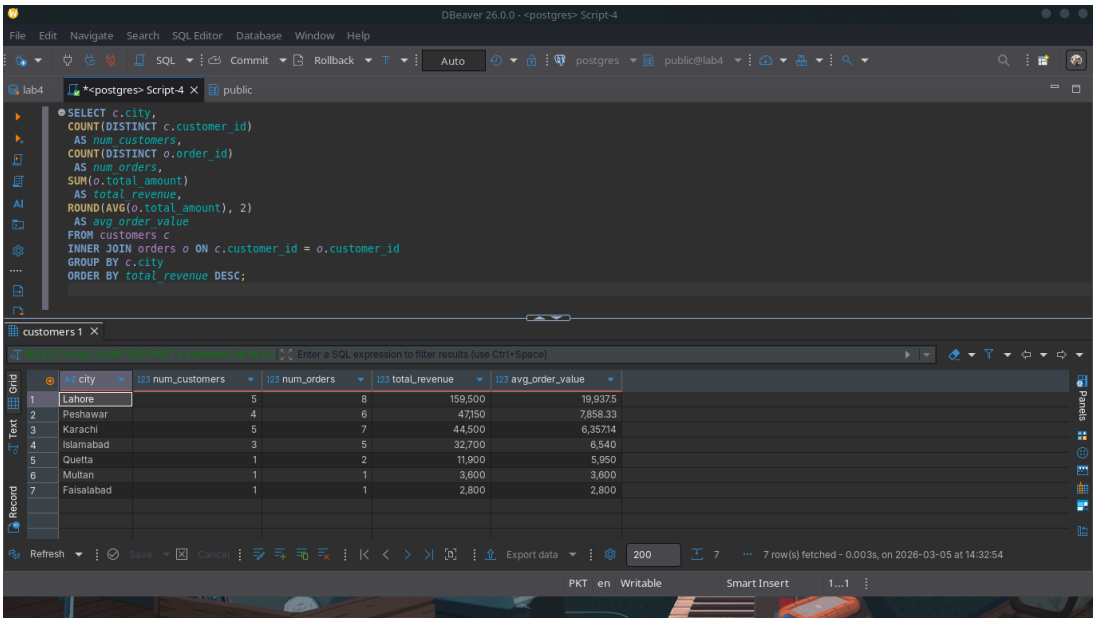

Figure 8: Query 7 Results: City-Level Sales Report

Results & Interpretation

Interpretation: Geographic analysis reveals market penetration and regional preferences. COUNT(DISTINCT) prevents double-counting customers with multiple orders. High-revenue cities may deserve targeted marketing or physical store expansion. Low-revenue cities with many customers indicate low average spend — a pricing or product mix issue.

1.8. Query 8: Order Details with Status Filter

Query 8

Objective: Show delivered orders from 2025, grouped by customer with product diversity and spend metrics.

```sql
SELECT c.name                                AS customer,
       c.city,
       COUNT(DISTINCT p.product_id)          AS distinct_products_bought,
       SUM(oi.quantity)                      AS total_items,
       SUM(oi.quantity * oi.unit_price)      AS total_spent
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON oi.product_id = p.product_id
WHERE o.status = 'delivered'
  AND o.order_date >= '2025-01-01'
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_spent DESC;
```

Listing 9: Query 8: Order Details with Status Filter

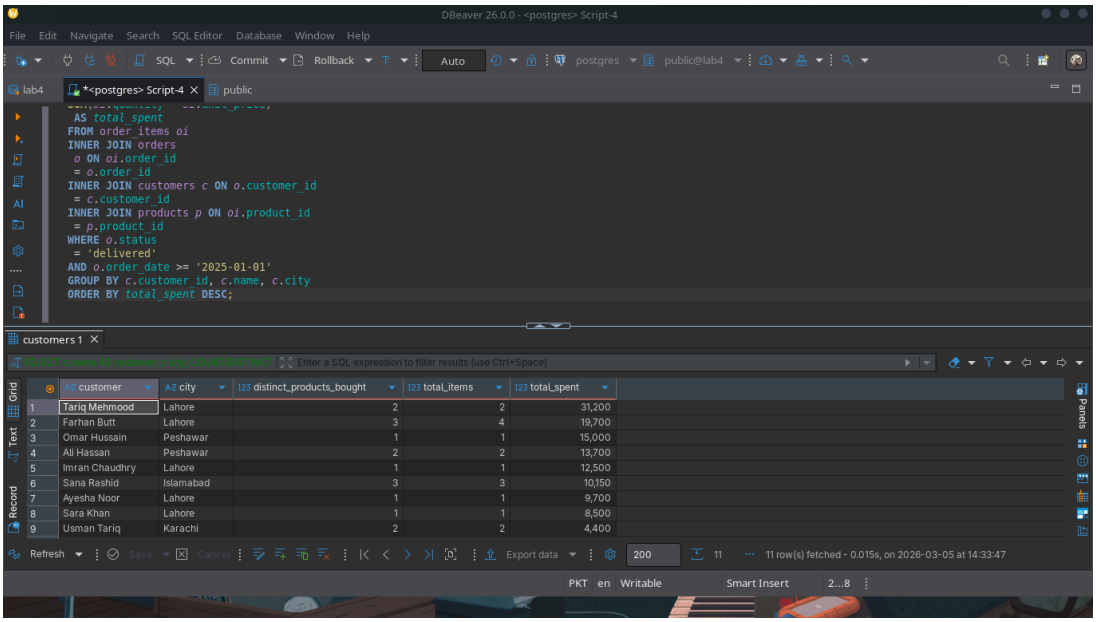

Figure 9: Query 8 Results: Delivered Orders from 2025

Results & Interpretation

Interpretation: This combines joins, WHERE filters, GROUP BY, and aggregates in one query. Filtering to 'delivered' status ensures we analyze completed transactions only. The distinct_products_bought metric shows product diversity — high values indicate customers exploring the catalog vs. repeat buyers of the same items.

1.9. Query 9: Session and Purchase Behaviour Together

Query 9

Objective: For every customer with sessions, show session count, orders, total spent, and conversion status.

```sql
SELECT c.name,
       c.city,
       COUNT(DISTINCT s.session_id)        AS total_sessions,
       COUNT(DISTINCT o.order_id)          AS total_orders,
       COALESCE(SUM(DISTINCT o.total_amount), 0) AS total_spent,
       CASE
           WHEN COUNT(DISTINCT o.order_id) > 0 THEN 'Converted'
           ELSE 'Not Converted'
       END                                 AS customer_status
FROM customers c
INNER JOIN user_sessions s ON c.customer_id = s.customer_id
LEFT JOIN orders o ON c.customer_id = o.customer_id
       AND o.status = 'delivered'
GROUP BY c.customer_id, c.name, c.city
ORDER BY total_sessions DESC;
```

Listing 10: Query 9: Session and Purchase Behaviour

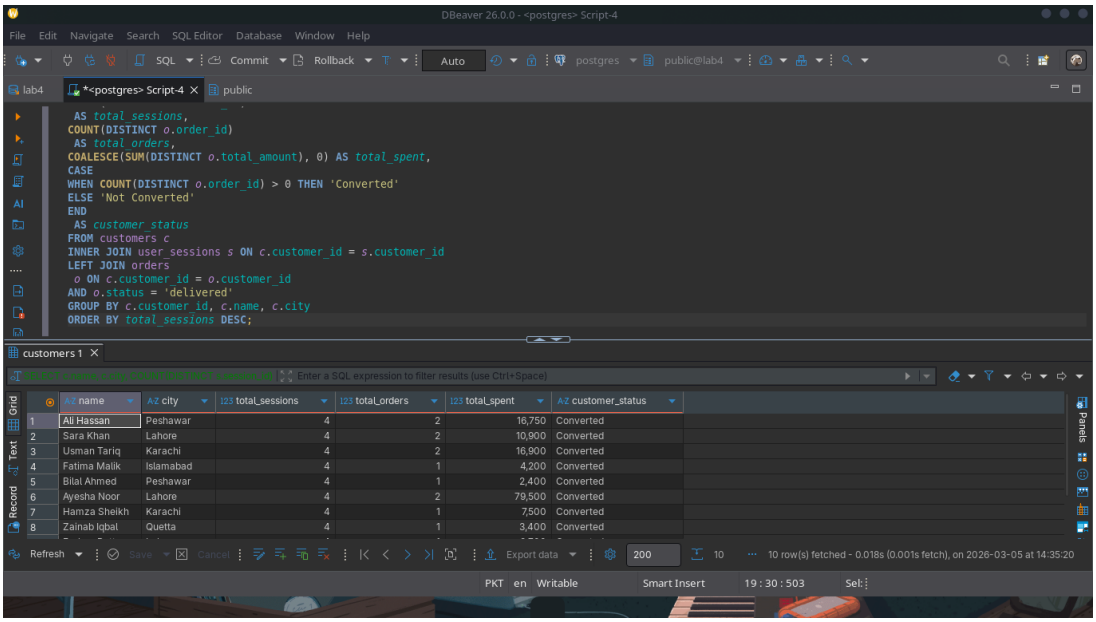

Figure 10: Query 9 Results: Session and Purchase Behaviour

Results & Interpretation

Interpretation: This query connects browsing behaviour (sessions) to purchases (orders). The filter on o.status is in the ON clause (not WHERE) to preserve the LEFT JOIN behaviour. High sessions with no conversion indicates UX/pricing problems. This is the foundation for conversion rate optimization.

1.10. Query 10: FULL OUTER JOIN — Data Completeness Audit

Query 10

Objective: Identify mismatches between customers and user_sessions tables (data quality audit).

```sql
SELECT c.customer_id                          AS customer_table_id,
       c.name,
       s.customer_id                          AS session_table_cust_id,
       COUNT(s.session_id)                    AS session_count,
       CASE
           WHEN c.customer_id IS NULL THEN 'Orphaned session      no
   customer record'
           WHEN s.customer_id IS NULL THEN 'Customer with no sessions'
           ELSE 'Matched'
       END                                    AS audit_status
FROM customers c
FULL OUTER JOIN user_sessions s ON c.customer_id = s.customer_id
GROUP BY c.customer_id, c.name, s.customer_id
ORDER BY audit_status, c.name;
```

Listing 11: Query 10: Data Completeness Audit

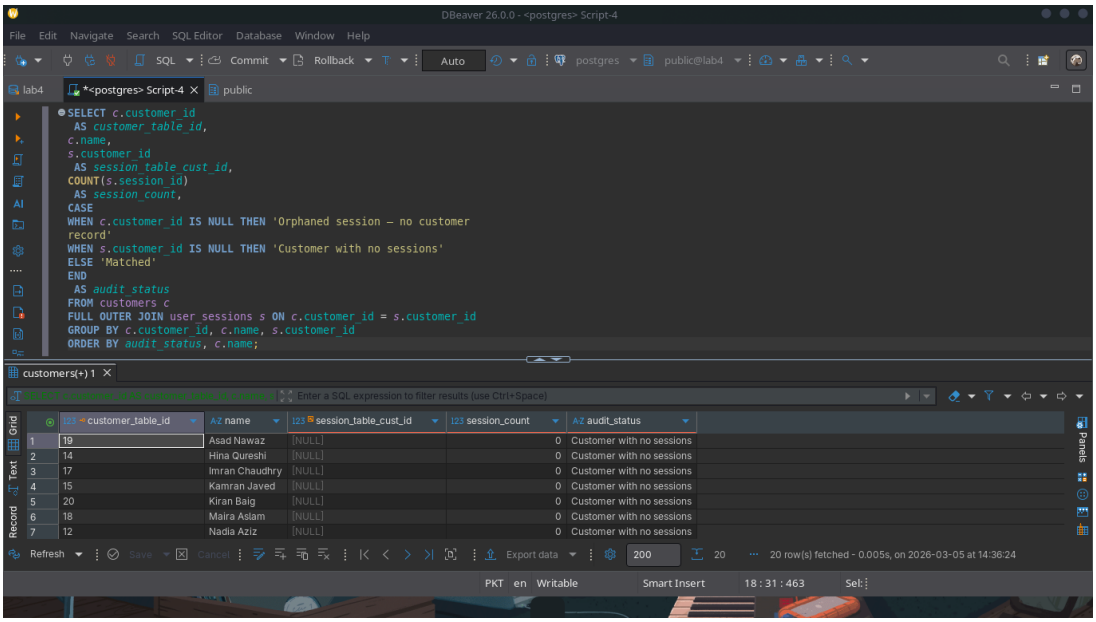

Figure 11: Query 10 Results: Data Completeness Audit

Results & Interpretation

Interpretation: FULL OUTER JOIN returns all rows from both tables, revealing data integrity issues. "Orphaned sessions" indicate tracking bugs or deleted accounts. "Customers with no sessions" may be legitimate (phone orders) or indicate tracking gaps. This audit should run before any analysis to ensure data quality.

Entity-Relationship Diagram (Text Representation)

ER Diagram

```
+------------------+       +------------------+       +------------------+
|    CUSTOMERS     |       |     ORDERS       |       |   ORDER_ITEMS    |
+------------------+       +------------------+       +------------------+
| PK customer_id   |<------| FK customer_id   |       | FK order_id      |
|    name          |  1:N  | PK order_id      |  1:N  | FK product_id    |
|    email         |       |    order_date    |------>|    quantity      |
|    city          |       |    status        |       |    unit_price    |
|    signup_date   |       |    total_amount  |       +------------------+
+------------------+       |    shipped_date  |                ^
         ^                 +------------------+                | 1:N
         |                                                     |
         | 1:N                                                 |
         |                                          +------------------+
         |                                          |    PRODUCTS      |
         |                                          +------------------+
         |                                          | PK product_id    |
         +------------------------------------------|    product_name  |
                     1:N                            |    category      |
                                                    |    price         |
+------------------+                                |    stock_qty     |
|  USER_SESSIONS   |                                +------------------+
+------------------+
| PK session_id    |
| FK customer_id   |
|    session_date  |
|    device        |
|    converted     |
+------------------+
```

Relationships:

- **customers → orders:** One-to-Many (one customer can place many orders)
- **orders → order_items:** One-to-Many (one order contains many items)
- **products → order_items:** One-to-Many (one product can appear in many order items)
- **customers → user_sessions:** One-to-Many (one customer can have many sessions)

Primary Keys (PK): customer_id, order_id, item_id, product_id, session_id

Foreign Keys (FK):

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id
- user_sessions.customer_id → customers.customer_id

2. ___Section 3 — Normalisation Reflection (15 points)

Option (b): Is the user_sessions table in 3NF?

Analysis: user_sessions Table Normalisation

The **user_sessions** table has the following structure:

- session_id (PK)
- customer_id (FK → customers.customer_id)
- session_date
- device
- converted

To determine if this table is in Third Normal Form (3NF), we verify three conditions:

1. First Normal Form (1NF)

The table satisfies 1NF because:

- Each column contains atomic (single) values
- There are no repeating groups
- Each row is uniquely identifiable by session_id
- No column contains multiple values like "device1, device2" in one cell

2. Second Normal Form (2NF)

The table satisfies 2NF because:

- The primary key is a single column (session_id), not composite
- All non-key columns (customer_id, session_date, device, converted) depend on the entire primary key
- There is no partial dependency issue since the key is not composite

3. Third Normal Form (3NF)

The table satisfies 3NF because:

- All non-key columns depend only on the primary key (session_id)
- There are no transitive dependencies
- customer_id is a foreign key, which is allowed in 3NF
- session_date, device, and converted all describe the session itself, not some other non-key attribute
- No column can be derived from another non-key column

Conclusion

Yes, the user_sessions table IS in Third Normal Form (3NF).
Justification:

- Every attribute depends on the primary key (session_id)
- The foreign key customer_id correctly references the customers table
- There are no derived columns (e.g., we don't store customer_name here)
- There are no transitive dependencies (e.g., device does not determine any other column)

3. Section 4 — AI Learning Log (20 points)

Entry 1: Debugging GROUP BY Error

Problem: Query returned error: "column must appear in the GROUP BY clause or be used in an aggregate function"

Wrong Query:

```sql
SELECT p.product_name, p.category, p.price,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
ORDER BY units_sold DESC
LIMIT 5;
```

AI Interaction: Asked why the error occurred.

Learning: When using aggregate functions (SUM, COUNT, AVG), ALL non-aggregated columns in SELECT must appear in GROUP BY. PostgreSQL is strict about this.

Corrected Query:

```sql
SELECT p.product_name, p.category, p.price,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.category, p.price
ORDER BY units_sold DESC
LIMIT 5;
```

Entry 2: INNER JOIN vs LEFT JOIN Confusion

Problem: Didn't understand when to use INNER vs LEFT JOIN for customer analysis.

AI Interaction: Asked for explanation of the difference with specific examples.

Learning:

- **INNER JOIN:** Only returns matching rows. Customers with no orders disappear.
- **LEFT JOIN:** Returns ALL left table rows. Non-matching right columns become NULL.
- Use LEFT JOIN when you need to include "zero" cases (customers who never ordered).

Entry 3: WHERE in ON Clause vs WHERE Clause

Problem: Query 9 was excluding non-converted customers unexpectedly.

Wrong Approach:

```sql
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status = 'delivered'  -- This turns LEFT JOIN into INNER JOIN!
```

AI Interaction: Asked why LEFT JOIN wasn't preserving all customers.

Learning: Filters on the RIGHT table of a LEFT JOIN must go in the ON clause, not WHERE. WHERE filters AFTER the join, eliminating NULL rows you wanted to keep.

Corrected:

```sql
LEFT JOIN orders o ON c.customer_id = o.customer_id
                      AND o.status = 'delivered'  -- Correct!
```

4. Section 5 — Reflection (10 points)

INNER JOIN vs LEFT JOIN: Understanding the Difference

The fundamental difference between INNER JOIN and LEFT JOIN is about **inclusion**. INNER JOIN is exclusive — it only returns rows where there's a match in both tables. If a customer has never placed an order, an INNER JOIN between customers and orders simply excludes that customer from the results. They become invisible. LEFT JOIN is inclusive — it returns ALL rows from the left table regardless of whether there's a match. Unmatched rows get NULL values for the right table's columns, but they appear.

I would use INNER JOIN when I only care about existing relationships: "Show me all orders with their customers" (I don't need orphaned orders). I would use LEFT JOIN when I need to find gaps or include zero cases: "Show me ALL customers with their order counts" (including customers with zero orders).

Most Surprising Discovery

The most surprising thing I discovered was how much insight is hidden in the **absence** of relationships. Looking at individual tables, I could see customers existed and orders existed. But only by joining them with LEFT JOIN and filtering for NULLs did I discover that some customers had never ordered. Similarly, products that had never been ordered only became visible through the anti-join pattern. This changed my perspective: the most valuable business insights often come from finding what's *not* there — customers who churned, products that don't sell, sessions that didn't convert. JOINs are not just about connecting data; they're about revealing the gaps that drive business decisions.

The FULL OUTER JOIN audit was particularly eye-opening. I had assumed the database would be perfectly consistent, but the possibility of orphaned records (sessions without customers, customers without sessions) revealed that real-world data pipelines have bugs. Data quality is not guaranteed — it must be audited.

Appendix: Performance Analysis (EXPLAIN ANALYZE)

Query 2 Execution Plan

```sql
EXPLAIN ANALYZE
SELECT c.name, o.order_id, o.order_date, o.status,
       p.product_name, p.category, oi.quantity, oi.unit_price,
       oi.quantity * oi.unit_price AS line_total
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
INNER JOIN customers c ON o.customer_id = c.customer_id
INNER JOIN products p ON oi.product_id = p.product_id
ORDER BY o.order_id, p.category;
```

Listing 12: EXPLAIN ANALYZE for Query 2

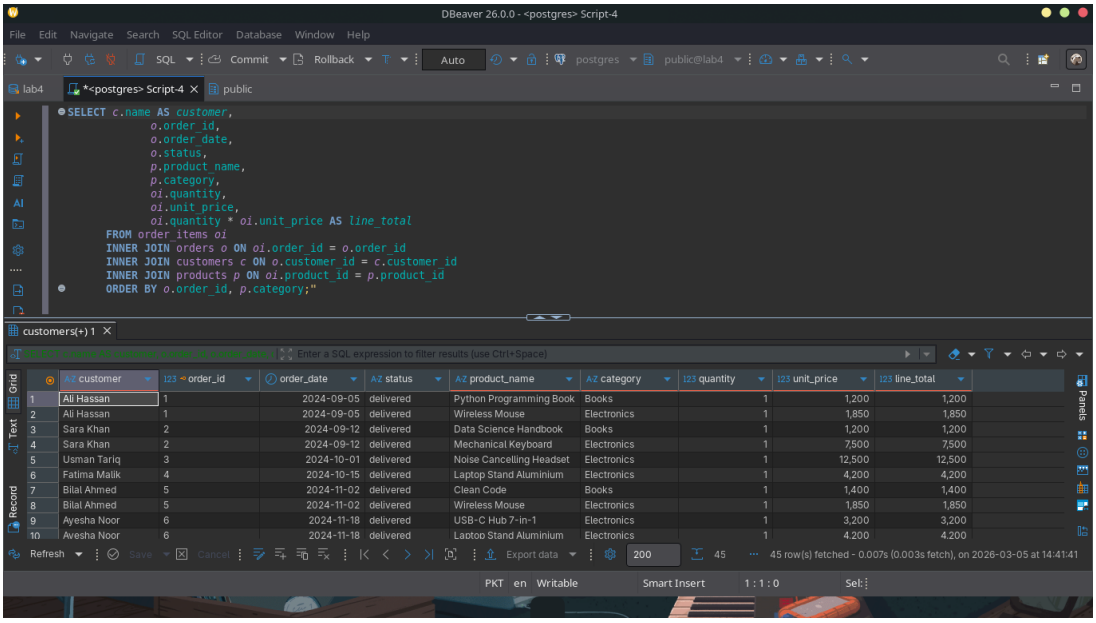

Figure 12: Query 2 Execution Plan

Operations Observed:

- **Sort:** PostgreSQL sorts the final result by order_id and category.
- **Hash Join:** Builds a hash table from one side and probes it for matches. Much faster than Nested Loop for large datasets.
- **Seq Scan:** Sequential scan reads entire tables (acceptable for small data).

On this small dataset, PostgreSQL chooses efficient hash joins. On millions of rows, it would still use hash joins but might add parallel execution and index scans.

References

[1] Afridi, M. U. (2025). *Database Systems Lab — Week 04 Manual: JOINs & Relational Data Modelling*. Institute of Management Sciences.